FIT2099 Assignment 3 Design Rationale

Requirement	Author
REQ 1: Second Spring	Poyee Tan 36641162
REQ 3: Infestation and Pest Control	Jingyu Zhang 37033956
REQ 4: Enchanted Tools	Yuanshu Liu 36226173
REQ 5: The Wanderer (API, HD)	Jingyu Zhang, Yuanshu Liu, Poyee Tan

Poyee Tan 36641162 – REQ 1: Second Spring

REQ1 makes plants alive. Every plant ages one turn at a time and tries its abilities (grow, spread, produce) in priority order, doing at most one thing per turn. Each young stage is its own class, and only tilled Soil accepts new plants.

A. One class per growth stage

Rejected 1: one class per crop with a `stage` field and a `switch` that picks the character, abilities and rules.

Pros	Cons
KISS: one class holds a crop's whole life.	OCP: a new stage edits every `switch` (the "switch statement" smell).
No new classes, so the class diagram stays small.	SRP: one class owns several display characters, ability lists and harvest rules.
	A sprout would still offer watering and harvesting, guarded by `if (stage == SPROUT)` checks.

Rejected 2: one class per crop holding a separate stage object (State pattern) that it swaps as it grows.

Pros	Cons
The crop keeps one identity on the tile while it grows.	The spec says growing creates "a brand new plant with age 0"; a swapped state would have to reset the age by hand (Connascence of Execution).
	The tile object would have to change its character, name and allowed actions at run time.

Chosen: one `Plant` subclass per stage. `Plant` is a new abstract class between `Ground` and `Crop`, and `Crop` now extends it. Growing replaces the ground with a new object, so the new stage starts at age 0 with no reset code. The young stages extend `Plant`, not `Crop`: as `Crop`s they would inherit watering and harvesting they must refuse.

Pros	Cons
OCP: a new stage is one new class. No existing stage changes.	Four small classes that mostly hold constants.
LSP: every `Crop` really can be watered and harvested; no sprout refuses a `Crop` method.	Stages name each other, e.g. `StrawberrySprout.nextStage()` returns `new Strawberry(...)` (Connascence of Name, kept inside `game.crops`).
SRP: each class has one character, one ability list and one set of rules.	

Package placement. The young stages live in `game.crops`, beside the crops they grow into, not in `game.plants`. Otherwise `game.plants` would import `game.crops`, which imports `game.plants`: a cycle (Acyclic Dependencies Principle). `game.plants` now depends on no crop.

Foresight: a Pumpkin Sprout that grows into a Pumpkin is one new class; no other class changes except `StackQueueValleyWorld`, which places it.

Connascence: Stages name their next stage (Connascence of Name), kept inside `game.crops`. The rejected State pattern would add Connascence of Execution: the age must be reset by hand on every swap. **Code smells:** Avoided: Switch Statements on a stage field, and Refused Bequest (a sprout inheriting watering and harvesting it must refuse). Accepted: small classes that mostly hold constants.

B. How a plant acts each turn

Every turn a plant ages, then tries its abilities in priority order. Three questions shape this: how the abilities are stored, how an ability gets what it creates, and how turns are counted.

Abilities as a list.

Rejected: each plant overrides `tick` with its own if-else.

Pros	Cons
KISS: each plant's rules sit in one place, with no shared class to learn.	DRY: "every N turns, roll a chance, stop at the first success" is copied into nine classes (Duplicate Code). A bug fix needs nine edits (Connascence of Algorithm).
	Easy to break the "one thing per turn" rule in one plant without noticing.

Chosen: `Plant` holds a `List<PlantAbility>` in priority order, passed to `super(...)`. `PlantAbility` has one method, `tryPerform(plant, location)`, which returns a report, or `""` when nothing happened. `Plant.tick` ages the plant, then `attemptGrowth` tries the abilities from the top and stops at the first non-empty report. `PeriodicAbility` implements the timing once: its `tryPerform` is `final` (Template Method). It checks `age % interval`, rolls the chance, then calls the abstract `perform`. `Grow`, `SelfReplicate` and `Produce` write only their effect.

Pros	Cons
DRY: the timing rule and the priority loop each exist once.	A reader opens two classes (the plant and the ability) to see one behaviour.
OCP: a new ability, e.g. "wither", is one new class.	Priority is the order of the constructor arguments (Connascence of Position). Each plant's class comment lists its abilities in priority order.
DIP: `Plant` depends only on `PlantAbility`.	`final tryPerform` stops an ability from changing the timing; an ability with different timing would implement `PlantAbility` directly.
SRP: `Plant` decides when to act; each ability decides what happens.	

`PeriodicAbility` rejects an interval below 1 or a chance outside 0 to 100 in its constructor; an interval of 0 used to crash on `age % 0`. `attemptGrowth` is `public` and does not age the plant, so REQ4's Bounty Enchantment reuses it to force an extra growth attempt instead of copying the loop (DRY across requirements).

Foresight: a Rose that withers after 30 turns is one new `PeriodicAbility` subclass placed first in its list.

Capabilities: `Growing`, `Spreading`, `Producing`.

Each ability must ask the plant what to create: `Grow` needs the next stage, `SelfReplicate` the offspring, `Produce` the item. The plant knows this, and already holds the `RandomUtils` and `AreaFactory` the new object needs.

Option	Pros	Cons
A `Supplier` per ability, e.g. `new Grow(..., StrawberrySprout::new)` (our first version)	The ability is given exactly what it creates.	The team removed method references and functional style everywhere after A2 feedback. The supplier also needed the plant's `RandomUtils` and `AreaFactory`.
Three methods on `Plant` whose defaults throw `UnsupportedOperationException` (our draft)	One abstract class; each plant overrides only the methods its abilities call.	ISP, Refused Bequest: Parsnip only produces, yet carried `nextStage()` and `offspring()` that could only throw. The Javadoc had to say "the default throws".
An abstract class per capability	Shared code could live in it.	Java has single inheritance: `Strawberry` already extends `Crop`, so it could not also extend a `ProducingPlant`.
One single-method interface per capability (chosen)	See below.	See below.

Chosen: `Growing` (`nextStage()`), `Spreading` (`offspring()`) and `Producing` (`produce()`). Each plant implements only what it can do. An ability finds its capability with `location.getGroundAs(Growing.class)` and does nothing if it is missing, the same way pests find `Regressible`.

Pros	Cons
ISP: no plant carries a method it cannot honour, and nothing throws (no Refused Bequest).	Three more interfaces, and every ability shares Connascence of Type with one of them.

Pros	Cons
No `instanceof`: `getGroundAs` is the engine's capability lookup.	A plant given `Grow` but not `Growing` silently never grows instead of failing. `PlantCapabilityTest` and the per-plant tests catch this.
Consistent with `Regressible` and `Plantable`, so a reader learns one pattern.	The capability methods must be `public`, as interface methods are.
DIP: abilities depend on three small interfaces, not on `Plant` subclasses.	

Under Design by Contract, the throwing draft was not strictly an LSP violation (no override weakened a postcondition, and no path reached the throw), so we describe it as an ISP problem.

Foresight: a Sunflower that drops seeds implements `Producing` and is given `Produce`; no ability changes.

Counting turns.

Option	Pros	Cons
The engine's `Status` countdown	Already exists.	Statuses tick and expire independently, so they cannot express "stop at the first success".
A global turn counter	One number for the whole game.	The engine has none; adding one would need a static field, a global-state smell A2 feedback penalised, and every plant would depend on its value (Connascence of Value).
The engine's `Behaviour<Plant, String>` as the ability type	No new interface.	Its contract returns `null` for "nothing" (Connascence of Meaning), and it is shared with actors; our contract is plant-specific and returns `""`.
`Plant` keeps its own `age` (chosen)	Each plant counts from 0 when placed, as the spec says; no setter, so only the plant changes it.	A plant spread right of or below its parent ages once in its first turn, because the engine ticks tiles row by row.

Randomness comes from a constructor-injected `RandomUtils`, so every test passes a `FixedRandom` and is deterministic.

Connascence: ability priority is the order of the constructor arguments (Connascence of Position), documented in each plant's class comment; it replaces Connascence of Algorithm across nine copies of the loop. Abilities share only Connascence of Type with the capabilities, where the rejected draft had Connascence of Meaning ("the default throws"). Ageing before trying abilities is Connascence of Execution, kept inside one method; a global counter would have spread Connascence of Value across every plant. **Code smells:** avoided Duplicate Code, Long Method (nine hand-written `tick` methods), global static state and functional style; removed Refused Bequest (methods that could only throw). Accepted: the Connascence of Position above, and the engine's row-by-row ticking, which shifts a new plant's timing by one turn.

C. Capabilities other features use: `Plantable` and `Regressible`

REQ1 offers two small interfaces to other features: `Plantable` says where a plant may go, and `Regressible` lets REQ3 pests set a plant back.

Where plants can grow.

Rejected: `instanceof Soil` wherever something is planted.

Chosen: a `Plantable` interface with `plant(ground, location)`, and a `PlantableTile` condition for `Area.filter`. `Soil` implements it. `plant` takes a `Ground`, not a `Plant`, so `game.positions` does not depend on `game.plants` (no package cycle).

Pros	Cons
OCP: a new plantable ground, e.g. fertilised soil, only implements the interface.	One more interface and one more condition class.

Pros	Cons
No `instanceof Soil`: type checks are the smell polymorphism replaces.	`plant` takes a `Ground`, not a `Plant`, so only Connascence of Type stops a caller planting something odd; it keeps `game.positions` free of `game.plants`.
DRY: `SelfReplicate`, the Merchant's `Crop.plantNextTo` and REQ5's Sowing omen all plant through it, so they follow one rule.	
Reused by REQ3: the mole and the snail find bare Soil through it.	

In Assignment 2 the Merchant could plant on Dirt; now it plants only on Soil, as the spec requires.

Foresight: a Greenhouse floor that accepts plants implements `Plantable`; every planting path accepts it at once.

Regressing, for REQ3 pests.

Rejected: pests call `Crop` methods directly.

Pros	Cons
KISS: no new interface to learn.	Pests would name every crop and stage (Connascence of Type on each); a new stage would edit every pest (OCP, Shotgun Surgery).

Chosen: `Plant` implements `Regressible`, whose `regress` replaces the plant with `previousStage()`. By default a regressed plant dies and leaves `Dirt`, matching A2's `Crop.removeCrop`; later stages override `previousStage()` to return their earlier stage.

Pros	Cons
DIP: pests depend on `Regressible`, so `game.pests` does not import `game.crops`; only Connascence of Name (`regress`) crosses packages.	A pest cannot treat one crop differently, so avoiding Feature Envy costs some flexibility.
OCP: a new plant is eaten by every pest with no change.	`previousStage()` is `protected`, so classes in the same package could call it. None does.

Connascence: only Connascence of Type on `Plantable` and Connascence of Name on `regress` cross package boundaries. `plant` takes a `Ground`, so `game.positions` does not depend on `game.plants` (no package cycle). **Code smells:** avoided type checks for `Soil`, Feature Envy on the pest side, and Shotgun Surgery when a new ground or a new plant is added.

D. Pausing and printing in a plant's turn

Two concerns sit around `Plant.tick`: REQ5's frost must stop a plant acting, and the plant must print what it did although `Ground.tick` returns nothing.

Pausing growth, for REQ5.

Option	Pros	Cons
The status removes the plant's abilities and puts them back (push)	The plant needs no check.	The status would reach into `Plant`'s private list and must restore it exactly (Connascence of Identity).
The plant checks for `GrowthPausedStatus` (pull, chosen)	Deciding whether to act stays the plant's job (SRP); the status only counts down.	`applyEffect` is empty on purpose, which a reader may see as a do-nothing override (it looks like Refused Bequest); a comment says why.

While paused, the plant still ages. REQ5's Frost omen applies the status; `TimedStatus` (A2) supplies the countdown, so no timer code is repeated.

Printing without a Display parameter.

`Ground.tick` returns nothing, so a plant prints its own report.

Option	Pros	Cons
`private static final Display` (our draft)	One shared object.	Shared static state, a global-state smell A2 feedback penalised.
A `Display` injected through the constructor	Strict DIP, and testable output.	Nine constructors, every `nextStage()` and `offspring()`, `StackQueueValleyWorld` and every test gain a parameter, yet `Display` is the engine's console writer with no alternative to swap in.
Return the report and let someone else print it	No `Display` in `Plant`.	Nothing calls `Ground.tick` except the engine, which ignores any result.
`private final Display` per plant (chosen)	No static state; KISS.	Each plant creates a `Display`, a small DIP gap; plants share nothing, so there is no Connascence of Value between them.

The collaborators that do vary, `RandomUtils` and `AreaFactory`, are injected.

Connascence: avoided Connascence of Identity (the rejected design had the status remove the plant's abilities and put the same list back) and Connascence of Value between plants (no shared static `Display`). **Code smells:** avoided a static field (global state). Accepted: an empty `applyEffect`, which looks like Refused Bequest and is explained by a comment, and a small DIP gap, because `Display` has no alternative to swap in.

E. Visibility, `final`, `protected` and `static`

Choice	Where	Why	Cost
Every field is `private`	All REQ1 classes	Implementation details stay inside (A2 feedback), so no class is a Data Class others manipulate, and connascence on state stays inside each class.	Each plant keeps its own `RandomUtils` and `AreaFactory` to pass to the plants it creates.
Constants are `private static final`	e.g. `GROW_INTERVAL`, `PRICE`	A name explains each number; Java does not allow instance fields in `super(...)`.	Tests repeat the numbers.
No static methods and no mutable static fields	—	A2 feedback penalised static methods.	—
`protected` constructors	`Plant`, `PeriodicAbility`	They are abstract; only subclasses construct them.	—
`protected` hooks	`perform`, `previousStage`	Subclasses override them; callers must go through the template.	Same-package classes could call them; none does.
`final` methods	`tryPerform`, `plantNextTo`, `noRoomToPlant`	Fix the shared rule so a subclass cannot change it.	A plant needing different timing implements `PlantAbility` directly.
`public` capability methods	`nextStage`, `offspring`, `produce`	Interface methods must be public.	Any class could ask a plant for its next stage; only the abilities do.
Classes are not `final`	e.g. `Grow`, `Soil`	Open for extension (OCP).	Private fields and `final` templates limit the damage.

Connascence: `private` fields keep connascence on state inside each class. **Code smells:** Avoided: public attributes and missing modifiers.

F. Changes to Assignment 2

Change	Why	Cost
`Crop` extends `Plant` instead of `Ground`	Every crop gets an age, abilities and `Regressible`.	`Crop` constructors take abilities. Watering and harvesting are unchanged.

Change	Why	Cost
`Unplanted` checks for any `Plant`	Watering a Cauliflower used to flood sprouts into puddles.	—
The Merchant plants through `PlantableTile`	The spec says only Soil accepts plants.	A Merchant surrounded by Dirt cannot sell crops; the message says so.
`RandomUtils` and `AreaFactory` are injected instead of static	A2 feedback; tests pass `FixedRandom`.	Every plant constructor has two more parameters ("tramp data" smell).
Fixed farm layout with one of every plant	Every plant is visible when the game starts.	—
Unique characters: Crow `b`, Cauliflower `K`	The spec gives `v` to the Strawberry Sprout, and `C` is the Cow.	Players of A2 see new characters.

G. Known minor violations and why we accept them

Violation or cost	Why it is acceptable
`SelfReplicate` and `Crop.plantNextTo` repeat the two lines that find an adjacent Plantable tile and plant on it. DRY is not fully met.	They build different messages. A third use would be the signal to extract it.
`Crop` also builds the Merchant's "no room to plant" message, mixing trading into a ground (SRP).	It keeps one wording for both purchasable crops, Tomato and Parsnip.
Each plant creates its own `Display` (Section D).	Injecting it adds parameters everywhere without making anything replaceable (KISS).
A plant given an ability without the matching capability does nothing (Section B).	Safer than a crash; tests check every plant's abilities.
The engine ticks tiles row by row, so a plant spread right of or below its parent ages once in its first turn.	Engine behaviour; it shifts timing by one turn and breaks no rule.
`StackQueueValleyWorld` registers the map characters with method references, `Dirt::new` (base code) and `Soil::new` (REQ1), although the team avoids functional style.	The engine's `DefaultGroundCreator.registerGround(char, Supplier<? extends Ground>)` needs a `Supplier`, and the engine cannot be changed. A named class that implements `Supplier` for each ground would be a class used once, only to avoid one line (KISS). These are the only two method references in the game code.

Testing

`GrowTest`, `SelfReplicateTest`, `ProduceTest`, `PlantStateTest`, `PeriodicAbilityTest` and `PlantCapabilityTest` cover every plant at the right age, one turn early, with a failed chance, with no free tile, while paused, when regressed, with invalid intervals and chances, and with an ability on a plant that lacks the capability. `FixedRandom` makes them repeatable, and `mvn test` runs them all.

Summary

Principle	Where it shows	Change it absorbs	Where it is only partly met, and why
SRP	Timing, each effect, planting, regressing and pausing each have one class.	A new timing rule edits only `PeriodicAbility`.	`Crop` holds one Merchant message (Section G).
OCP	A new stage, ability, capability or plantable ground is one new class.	A Pumpkin, a wither ability, a Greenhouse floor: no existing class changes.	New plants are placed in `StackQueueValleyWorld`, the composition root.

Principle	Where it shows	Change it absorbs	Where it is only partly met, and why
LSP	Every `Crop` can be watered and harvested; abilities never check the plant's type.	A new stage works wherever `Plant` is expected.	—
ISP	`Growing`, `Spreading`, `Producing`, `Plantable`, `Regressible` and `PlantAbility` each have one method.	Parsnip carries only `produce()`.	—
DIP	Abilities depend on capabilities, pests on `Regressible`, `Plant` on `PlantAbility`; `RandomUtils` and `AreaFactory` are injected.	A new plant or pest needs no change to the other side.	Each plant creates its own engine `Display`; stages create their next stage directly (Connascence of Name, one outcome each, so a factory would be a parameter nobody varies).

DRY and KISS. Each rule exists once: the timing, the priority loop, planting and the countdown. We added no layer that only one class uses. **Connascence.** Between packages it is only of Name and Type, through `Regressible`, `Plantable` and the capability interfaces. Stronger forms (Position for ability priority, Execution for age before abilities) stay inside one class each.

Jingyu Zhang 37033956 – REQ 3: Infestation and Pest Control

Three pests, `SlimeSnail`, `LocustSwarm` and `BurrowingMole`, implement `Infestation`: they move towards plants, set them back a growth stage, and each leaves its own mark on the farm. Three defences, `PestTrap`, `LadybugJar` and `SaltBarrier`, implement `PestDeterrent`: they pull pests in, hunt them, or push them out.

A. Two abstractions, found by capability, never by class

REQ3 has two sides that must not know each other's concrete classes: pests eat plants, and defences act on pests. Both sides find what they need by asking for a capability.

Rejected: `Infestation` and `PestDeterrent` as abstract classes.

Pros	Cons
KISS: one type fewer on each side, and shared code could live in the abstract class.	Java has single inheritance. `Pest` already extends `WanderingActor` (from A2) and `DeterrentItem` extends the engine's `Item`, so neither could also extend an abstract `Infestation` or `PestDeterrent`.

Finding the other side. Defences must tell a pest from the Farmer, a cow or a ladybug, and pests must tell a plant from bare ground.

Option	Pros	Cons
`instanceof Pest` in each defence	Short, and familiar to every Java reader.	A type check is a code smell, and each defence names the pest class (Connascence of Type on `Pest`). A pest that is not a `Pest` subclass would be missed.
A list of pest classes	Explicit: every pest is named in one place.	Every new pest edits the list (OCP).
Pests call each crop's methods	KISS: no interface between pests and plants.	Pests would name every crop and stage (Connascence of Type on each); a new crop would edit every pest (OCP, Shotgun Surgery).
Capability lookup (chosen)	DRY and OCP: one check per side, and a new pest or plant is found with no edits.	Two small pieces to learn, `PestPresent` and the `Regressible` lookup, instead of one direct type check; a reader must know that a capability, not a class, decides who is a pest.

Chosen: both abstractions are one-method interfaces, and both sides ask for capabilities. Defences use `PestPresent`, a `TileCondition` (the area filter from A2) that checks `actor.asCapability(Infestation.class)`. Pests use `getGroundAs(Regressible.class)`, from REQ1, and call `regress`. A pest is an actor and an `Infestation` at once.

Pros	Cons
DIP: defences never name `Pest`, and pests never name a crop, so `game.pests` does not import `game.crops`.	A pest cannot treat one crop differently, for example a snail that prefers Strawberries; that would need a new method on `Regressible`, which every plant would then implement.
OCP: a new pest is found by every defence, and a new plant is eaten by every pest, with no edits (no Shotgun Surgery).	`PestPresent` is a very small class, close to a Lazy Class; three users (trap, salt, ladybug) justify it.
ISP: `Infestation` and `Regressible` each have one method, and defences never see a pest's movement or health.	Interfaces hold no shared code, so the common pest turn needs its own abstract class (Section B).

Foresight: a cursed scarecrow that is not a `Pest` could implement `Infestation`, and every trap, jar and salt bag would find it unchanged.

Connascence: between packages, only Connascence of Name (`feedOn`, `regress`) and of Type (`Infestation`, `Regressible`). No class depends on `SlimeSnail`, `LocustSwarm`, `BurrowingMole` or any crop. **Code smells:** avoided type checks (`instanceof`), Shotgun Surgery and Feature Envy (pests reaching into crop internals). Accepted: `PestPresent` is close to a Lazy Class.

B. One pest turn: feed or move, with movement as data

Rejected: one `Pest` class with a `PestKind` enum and a `switch` in `feedOn`.

Pros	Cons
KISS: one file holds every pest.	OCP: a new pest edits the switch, and every other switch on the enum.
A reader sees all pest rules side by side, in one file.	SRP: one method mixes slime, area damage and cave-ins. This is the "switch statement" smell.
	Each case would need its own fields (meals left only for the swarm), so the class carries fields most pests never use.

Chosen: `Pest` is an abstract class (Template Method). Its turn is written once: if the pest can feed here, it returns an `InfestAction`; otherwise its `SeekPlantBehaviour` moves it. A subclass gives only its landing offsets and its `feedOn`. One hook, `canFeedAt(Location)`, is true when a plant is underfoot. `LocustSwarm` overrides it, because a swarm also feeds on the plants next to it.

Pros	Cons
OCP: a new pest is one new class. `Pest`, `InfestAction` and the defences do not change.	A reader must open two classes to see one pest's full turn; subclasses share Connascence of Name with `Pest` through `feedOn` and `canFeedAt`.
LSP: the turn never asks which pest it is. The hook replaces `if (this instanceof LocustSwarm)`, which would make the parent know its children.	A subclass could override `canFeedAt` badly, for example return true where `feedOn` finds nothing. `feedOn` then returns an empty report, so the game does not break.
DRY: "feed or move" exists once. `Pest` extends the existing `WanderingActor`, so the unconscious check from A2 is reused, not copied.	

How a pest moves.

Option	Pros	Cons
Each pest overrides `move()`	Easy to read: each pest's movement sits in its own class.	DRY: "find the nearest plant, pick a landing, fall back" is copied three times. A bug fix needs three edits (Connascence of Algorithm).
Three classes (`KnightMove`, `CrawlMove`, `BurrowMove`)	Each move has a name a reader can search for.	Three classes that differ only in a table of numbers. The A2 marker called one class per variant "architectural bloat".
One `SeekPlantBehaviour` with offsets (chosen)	DRY: one copy of the search. OCP: a new movement is one new array. KISS: no new type per pest.	The offsets are a raw `int[][]`, a mild "primitive obsession" smell.

We accept the smell because the array never leaves its pest class, and the behaviour validates and copies it in its constructor. So a pest cannot pass `{1}` or change the table later. The fallback is typed as the engine's `Behaviour`, not `WanderBehaviour` (DIP). The pest lands on a tile and does not walk through the tiles between, so a jumping pest can pass a wall or a salted tile. Two rules came from running the game: a mole that only moved two tiles could never reach a plant one tile away, so it now moves one or two; a knight jump rarely lands on a plant, so the swarm feeds from the tiles next to it. Unit tests cover both.

Why `feedOn` is not a template. We considered a template on `Pest` where each pest gives only a per-tile step. But the three effects touch different tiles (the area around the swarm, the mole's four straight neighbours, the bare ground around the snail), do different things to them, and only the swarm changes itself (meals, splitting). A template would wrap three different effects without removing repeated code (KISS).

Foresight: an Aphid that hops three tiles and poisons the soil is one class: a new offset table and a new `feedOn`. A grasshopper that leaps three tiles in a straight line is one new offset table; `SeekPlantBehaviour` stays the same.

Connascence: `InfestAction` knows a pest only through `Infestation` (Connascence of Name). Connascence of Algorithm between three copies of the search is avoided. The offsets carry Connascence of Meaning (each

row is a `{dx, dy}` pair), but only `SeekPlantBehaviour` reads them, and its constructor rejects rows that are not pairs. **Code smells:** avoided Switch Statements, Duplicate Code, one class per variant ("architectural bloat") and Speculative Generality (a template for effects that share nothing). Accepted: Primitive Obsession (`int[][]`), local and copied defensively; `InfestAction` is close to a Middle Man, because the engine needs an `Action` object for every turn.

C. One way to use every defence

Rejected: three actions, `SetTrapAction`, `OpenJarAction` and `ThrowSaltAction`.

Pros	Cons
Each action can have its own menu text and its own class name.	DRY: each repeats "remove the item, act, report" (Duplicate Code). If the rule for using an item changed, three classes change (Connascence of Algorithm).
	OCP: each new defence also needs a new action.

Chosen: `DeterrentItem` is an abstract item. While the Farmer carries it, it offers one `DeployDeterrentAction`, which calls `deterAround` and removes the item. The action holds a `PestDeterrent`, not a `PestTrap` (DIP). The hook `canDeployAt(Location)` hides the option where it cannot work: a trap is not offered on a plant, so a failed attempt never wastes the item.

Pros	Cons
OCP/DIP: a new defence is one subclass. The action never changes.	The action keeps two fields for one object: the `Item` (the engine's inventory only removes an `Item`) and the `PestDeterrent`. They must be the same object (Connascence of Identity). `DeterrentItem` passes `this` twice in one place, so callers cannot mix them up.
SRP: the item says what it does; the action says how an item is used, so each changes for one reason.	

Rejected fixes for the two fields: a cast breaks type safety; a generic `<T extends Item & PestDeterrent>` adds a type parameter that only removes one field.

Foresight: a Scarecrow that scares pests away for ten turns is one `DeterrentItem` subclass plus one `TemporaryGround` subclass.

Connascence: the action's two fields must be the same object (Connascence of Identity), kept local in one line of `DeterrentItem`. **Code smells:** avoided Duplicate Code and a parallel hierarchy of one action per item. Accepted: two fields for one object, which is cheaper than a cast.

D. Temporary ground and the three defences

Slime, salt and the trap replace a tile's ground for a while and then put the old ground back. The trap and the salt are each an item and a ground. This looks like a "parallel hierarchy" smell, but the engine keeps carried things (`Item`) and map tiles (`Ground`) as separate types, and one object cannot be both: the item lives in the inventory; the ground lives on the map with its own timer.

Rejected: each ground keeps its own counter.

Pros	Cons
KISS: no abstract class to learn; each ground reads alone.	DRY: the same countdown in three classes (Duplicate Code, and Shotgun Surgery for any timing change). A change such as "rain shortens every temporary ground" edits three classes.
	A ground can forget to restore the old ground, a bug the compiler cannot catch.

Chosen: `TemporaryGround` is abstract. Its `tick` is `final`: run `tickEffect` (empty by default), stop if the effect already replaced this ground, then count down and restore the original ground.

Pros	Cons
The order "effect, check, count down" is fixed in one place (Connascence of Execution kept local).	`final` stops a subclass from changing the countdown itself, for example a ground that never expires. Such a ground would extend `Ground` directly.
KISS: `SlimeGround` and `SaltGround` are only constants, so a new ground is a few lines.	

Package placement. `TemporaryGround` sits in `game.positions`. `SlimeGround` sits in `game.pests`, and `SaltGround` and `TrapGround` in `game.pestcontrol`. If all three were in `game.positions`, that package would import `game.pests`, which already imports `game.positions`: a cycle. Each ground sits beside the feature that creates it (Acyclic Dependencies Principle).

PestTrap and TrapGround (pull in). Each turn the trap pulls every pest within two tiles one step closer, then captures pests on or next to it, up to two. Grounds tick before actors move. In our first version the trap captured first; a pulled mole then walked away on its own turn, and the trap never caught it. Pulling first fixed this (Connascence of Timing, explained in the class comment and locked in by a test). A ground's `tick` returns nothing, so the trap cannot hand its report to the engine as an action does.

Option	Pros	Cons
A `static Display`	One shared object for every trap in the game.	A static field: global state, the smell the A2 marker penalised.
A `private final Display` field, as `Plant` does	Matches REQ1, so both features print the same way.	Every trap holds a `Display` it rarely uses, and the diagram gains an association with no state.
A `Display` passed into the constructor	Testable: a test could pass a recording `Display`.	The `Display` would travel through the item and the action, which have nothing to print.
Make the trap an actor	Uses the engine's normal message path, like every other actor.	A trap is not a creature; it changes the TA-approved design.
A local `Display` when there is news (chosen)	No static state; a dependency, not an association.	Tests check the map, not the printed text; the trap shares only Connascence of Name (`println`) with `Display`.

LadybugJar and Ladybug (hunt). The jar releases up to three ladybugs around the Farmer. A ladybug eats a pest next to it, or wanders. It loses one hit point a turn, so it lives five turns, and the existing `ConsciousActor` removes it at zero (DRY). The cost: lifespan and health are one number. If something healed a ladybug, it would live longer. Nothing in the game does.

SaltBarrier and SaltGround (push out). The bag salts bare Soil and Dirt around the Farmer, burns any pest on those tiles and pushes it out. All of this happens once, inside `deterAround`. After that, `SaltGround` only refuses pests in `canActorEnter` (KISS).

Connascence: "effect, check, count down" is Connascence of Execution, kept inside one `final` method. Grounds ticking before actors move is Connascence of Timing, kept inside `TrapGround` and locked in by a test. Placing each ground beside its feature keeps package dependencies one way. **Code smells:** avoided Duplicate Code, Shotgun Surgery and a static `Display` (global state). Accepted: `final` stops a subclass from changing the countdown, and the item-and-ground pairs look like a Parallel Inheritance Hierarchy for the engine reason above.

E. Visibility, `final`, `protected` and `static`

A2 feedback said package-private helpers (`CrowSensing`, `CrowNavigation`) can be good encapsulation, but the rationale must say so. Here is each choice in REQ3.

Choice	Where	Why	Cost and when we would change it
Every class is `public`	All 20 REQ3 types, including the small `PestPresent`, `InfestAction` and `EatPestAction`	A3 is the last assignment, so package-private would be safe today. We still chose `public`, because extensibility is a design goal and other features already live in other packages. A defence in another package, for example a frog in `game.actors`, should reuse `PestPresent` and `EatPestAction` without moving them.	A wider public API. We considered package-private for `InfestAction` (only `Pest` uses it) but chose one rule for the whole feature, so a reader never wonders whether a hidden class was forgotten.
Every field is `private`	All classes	Implementation details stay inside (A2 feedback). Subclasses use hooks, not fields, so connascence on state stays inside each class.	`SlimeSnail`, `BurrowingMole` and `LocustSwarm` each keep their own `AreaFactory`, although `Pest` also receives it. A `protected` getter on `Pest` would remove three fields but expose a tool that `Pest` itself does not use.
Constants are `private static final`	e.g. `LocustSwarm.SPLIT_CHANCE_PERCENT`, `SlimeSnail.HIT_POINTS`	A fixed rule belongs to the class, not to one object. A name such as `SPLIT_CHANCE_PERCENT` explains the number, so there are no magic numbers. Java does not allow instance fields in `super(...)`, so names, characters and hit points must be static to be named.	A test cannot read them, so tests repeat the number.
No static methods, no `public` constants and no mutable static fields	—	This is what the A2 marker penalised: static methods are "function thinking", and `public` constants leak details. A `private static final` constant does neither: it cannot change and is not seen outside the class.	—
`protected` constructors	`Pest`, `DeterrentItem`, `TemporaryGround`	They are abstract; only subclasses construct them.	—
`protected` hooks	`canFeedAt`, `canDeployAt`, `tickEffect`	Subclasses must override them; callers must not skip the template.	Classes in the same package can also call them. No class does.
`protected final`	`TemporaryGround.revertAt`	A subclass may end its ground early (the trap after two captures) but cannot change how it ends.	—
`final` method	`TemporaryGround.tick`	Fixes the order of the countdown (Section D).	See Section D.
Classes are not `final`	e.g. `LadybugJar`, `SaltGround`	Open for extension (OCP), e.g. a larger jar.	A subclass could break a rule. Private fields and the final `tick` limit the damage.

Choice	Where	Why	Cost and when we would change it
`private` constructor	`LocustSwarm(int meals, ...)`	Only a split may create a smaller swarm. The public constructor always gives four meals, so a half-empty swarm cannot be built by mistake.	A future "hungry swarm" difficulty would need a new public constructor.
`private` helpers	e.g. `pullPests`, `splitAt`, `copyOf`	They are steps, not contracts. Making them `protected` would freeze their signatures for every subclass.	A subclass that wants a different pull must override `tickEffect`.
Non-final fields	`mealsLeft`, `capturesLeft`, `remainingTurns`	These values change every turn.	—

Explicit modifiers, and why we keep `protected`. Two pieces of advice pull in different directions. The Week 9 lecture and our tutor suggest a narrow API: keep a class package-private when no other package needs it, and try not to use `protected`, because in Java it also opens a member to every class in the same package. But Java has no keyword for package-private: it means writing no modifier at all, and the A2 marker penalised exactly that ("missing modifier" on `CrowSensing` and `ForageableItems`).

Option	Pros	Cons
Package-private classes and hooks (no modifier) where every user is in the same package	The narrowest API; fewer dependencies across packages.	A reader cannot tell a deliberate choice from a forgotten modifier, which A2 penalised. A pest or defence in another package could not extend `Pest` or `DeterrentItem`.
Replace the template hooks with strategy objects, so no `protected` is needed	No `protected` at all.	A new interface and class for each hook (`canFeedAt`, `canDeployAt`, `tickEffect`) only to avoid a modifier: needless complexity (KISS).
Explicit `public` classes and `protected` hooks (chosen)	Every class and member states its access, so nothing looks forgotten. `protected` is used only for constructors of abstract classes and for hooks that subclasses override; no field is `protected`.	A wider API than strictly needed, and classes in the same package could call a hook. None does, and the template methods that call the hooks are `final` or fixed in the parent, so a hook cannot skip the rules.

Connascence: `private` fields keep connascence on state inside each class. `protected` hooks create connascence only with subclasses, never with callers. **Code smells:** Avoided: public attributes, which make a Data Class that others manipulate, and missing modifiers. Accepted: a wider API than strictly needed (see the table above).

F. Changes to the earlier design

REQ3 is new in A3. The only earlier class it changes is `StackQueueValleyWorld`, which places the pests and gives the Farmer the defences.

Statics replaced by injection. REQ3's first version called the A2 static helpers `Area.within`, `Area.adjacentTo` and `RandomUtils.rollPercent`. The A2 marker called static helpers "function thinking". Following that feedback, the team now creates one `RandomUtils` and one `AreaFactory` in `StackQueueValleyWorld.initialise`, and every REQ3 class that needs them receives them through its constructor.

Pros	Cons
DIP: a pest depends on the objects it is given, not on global classes. Tests pass `FixedRandom` directly.	Every pest and defence constructor gained one or two parameters, and `Pest` must pass them on (a "tramp data" smell).
No hidden global state, so two tests cannot affect each other.	Connascence of Position in the constructors. Each has at most two such parameters, so the order is easy to check.

Display character. `SaltBarrier` first used `k`, which clashed with the A2 `CauliflowerItem`. Earlier characters are fixed, so the new class changed to `f`.

Connascence: Connascence of Position in the constructors, with at most two such parameters. **Code smells:** Avoided: global static state. Accepted: Tramp Data and a small Data Clump, because the same two objects travel together through constructors.

G. Known minor violations and why we accept them

Violation or cost	Why it is acceptable
`SlimeSnail` and `SaltBarrier` both ask "is this tile bare Soil or Dirt?" (two lines each). DRY is not fully met.	Rule of three: two short copies are cheaper to read than a new class. A third use would be the signal to extract a `BareFarmGround` condition.
`LadybugJar` catches `GameEngineException` and moves on. An empty catch is a smell.	The tile is checked with `canActorEnter` first; the catch only covers a tile taken in between. A comment says so.
`Ladybug`, `LadybugJar` and `SaltBarrier` use `getExits()`, while other classes use `AreaFactory`.	`getExits()` is the engine's own list of neighbours and needs no extra dependency. `AreaFactory` is used where we filter or search an area.
`TrapGround` moves actors, which is more than most grounds do.	Acting on the pests around it is the trap's whole job. Only the trap does this each turn, and the logic stays in one class.
Pests (except the swarm) never leave on their own.	On purpose: the defences are the counter to the pests. A meal limit could be added in `feedOn` only.

Summary

Principle	Where it shows	Change it absorbs	Where it is only partly met, and why
SRP	Moving (`SeekPlantBehaviour`), the turn (`Pest`), finding pests (`PestPresent`), using an item (`DeployDeterrentAction`) and timing ground (`TemporaryGround`) each have one class.	A new landing rule edits `SeekPlantBehaviour` only; a new countdown rule edits `TemporaryGround` only.	`TrapGround` pulls, captures and prints. `LocustSwarm` feeds, splits and leaves. Each is one rule from the brief, and splitting them would add classes used once.
OCP	A new pest, defence or temporary ground is one new class.	An Aphid, a Scarecrow or a Frost ground: no existing class changes.	The new class must still be placed in `StackQueueValleyWorld`. That is the composition root, so this is expected.
LSP	Every pest works wherever `Pest` or `Infestation` is expected; every defence wherever `PestDeterrent` is. Hooks replace type checks.	A new pest is fed, found and trapped with no new branch.	`canFeedAt` and `feedOn` must agree. If they do not, `feedOn` returns an empty report, so the game stays correct. `final tick` stops a ground from breaking the countdown.
ISP	`Infestation`, `PestDeterrent` and `Regressible` each have one method. Defences never see a pest's movement or health.	A pest that moves differently does not touch any defence.	`PestPresent` uses `Infestation` only to recognise a pest and never calls `feedOn`. This is not a marker interface: `InfestAction` calls `feedOn`.
DIP	`DeployDeterrentAction` depends on `PestDeterrent`, pests on `Regressible`, `SeekPlantBehaviour` on `Behaviour`; `RandomUtils` and `AreaFactory` are injected.	A new crop, defence or fallback behaviour needs no change to the classes that use it.	`Pest` creates `SeekPlantBehaviour` and `WanderBehaviour`; `SlimeSnail` creates `SlimeGround`; `LadybugJar` creates `Ladybug`. Each has one outcome, so an injected factory would be a parameter nobody varies (YAGNI).

DRY and KISS. Each rule exists once: the pest turn, the landing search, finding a pest and the countdown. We added no layer that only one class uses. **Connascence.** Between packages it is only of Name and Type. Stronger forms (Timing, Identity, Execution) stay inside one class each, with a comment and a test.

Testing. Every class has normal, edge and invalid tests. `FixedRandom` makes them repeatable, and `mvn test` runs them all.

Yuanshu Liu 36226173 – REQ 4: Enchanted Tools

The Farmer can enchant a Hoe, Watering Can or Pickaxe (retrofitted as `Enchantable`) with a Blaze, Bounty or Shatter Enchantment (new `Enchantment` classes). The next use of the tool fires its own spread and then the Enchantment on every tile the spread touched; the Enchantment is then spent.

A. Finding and firing the tool that was used, without `instanceof`

After tilling, watering or breaking, the action must fire the Enchantment of the exact tool it used. This needs a capability, a shared lookup and a way to recognise that one tool.

A capability, not a type check.

`CapabilityToolAction` (tilling and watering) and `BreakAction` must fire the Enchantment of the tool they just used.

Rejected: check the tool's class, for example `if (tool instanceof Hoe)`, and read its enchantment directly.

Pros	Cons
KISS: short, and no new interface or helper to learn.	OCP: every new enchantable tool adds a branch to two actions, which is the "switch statements" smell spread over two classes.
A reader sees in the action itself which tools can carry an Enchantment.	Connascence of Type between two actions and every tool class: renaming or adding a tool edits both actions (Shotgun Surgery).

Chosen: a capability interface, `Enchantable`, with `enchant(Enchantment)` and `triggerEnchantment(Actor, Location)`.

Pros	Cons
DIP: both actions depend only on `Enchantable`, so they compile without knowing any tool class.	One more interface to read, although it follows the existing `Tillable` and `Waterable` pattern.
OCP: a new enchantable tool needs no change to either action, and no existing test changes.	The actions still need a way to find the tool's `Enchantable` view, which the shared lookup below solves.
Encapsulation: the tool, which owns the Enchantment, decides whether it has anything to fire.	

One shared lookup.

`CapabilityToolAction` and `BreakAction` both need the same lookup after their own effect: find the tool instance just used among the actor's `Enchantable` items, and fire it.

Rejected: write the lookup loop inside each action.

Pros	Cons
KISS: no new class.	DRY: the same loop in two actions, in two unrelated class hierarchies (Duplicate Code).
The lookup sits next to the code that needs it, so each action reads alone.	If the lookup changes, both copies must change together (Connascence of Algorithm across classes, which Week 9 rates as needing care).

Chosen: `EnchantmentTrigger.fire(Actor, Object usedTool, Location)`, used by both actions. In the first version only `CapabilityToolAction` needed the lookup. When `BreakAction` needed the same loop, we moved it into one class, so it was created for a real second user, not "just in case".

Pros	Cons
SRP: one place knows how to go from a tool in hand to its `Enchantable` view.	The `usedTool` parameter is typed `Object`, so a wrong argument fails silently instead of at compile time; Week 10 would prefer generics (Section F).
OCP: a future action that uses a tool, such as a scything action, reuses it with no new code.	A new object is created on each use; it holds no state, so this only reads oddly.

Why `==` is used, and why it is not a hidden `instanceof`.

The engine's `Inventory.getItemsAs(Class)` returns views narrowed to one capability. A tool held as a `TillingTool` cannot be asked for a second capability, and the engine cannot be changed. So `EnchantmentTrigger` takes a second view, `getItemsAs(Enchantable.class)`, and looks for the entry that is the same object as the tool in hand (`enchantable == usedTool`).

Option	Pros	Cons
`instanceof Enchantable` plus a cast	Short, and familiar to Java readers.	Answers a different question ("what class is this?") and brings back the Connascence of Type that the interface above removed; a type check is the smell polymorphism should replace.
Compare with `equals`	Familiar, and reads like value comparison.	Tools do not override `equals`, so it would quietly mean `==` anyway; two equal-looking tools would also be confused (Connascence of Meaning).
Compare references with `==` (chosen)	It asks the real question, "is this the exact tool used this turn?", with no cast.	Connascence of Identity, the strongest kind; acceptable only because both references are found and compared inside one method and never stored.

Connascence: the actions share only Connascence of Type on `Enchantable` (instead of on three tool classes) and Connascence of Name on `fire`. The one Connascence of Identity (`==`) stays inside one method. **Code smells:** avoided type checks, casts, Switch Statements on the tool's class and Duplicate Code. Accepted: a weakly typed `Object` parameter. **Foresight:** an enchantable Scythe implements `Enchantable` through the base class in Section B, and the actions stay the same.

B. Sharing enchanting: one base class and one slot

All three tools store, offer, fire and clear Enchantments the same way; only their spread differs.

One base class.

All three tools must store the Enchantments they offer, offer one `EnchantAction` per Enchantment, and hold the one currently attached. None of this depends on the tool.

Rejected: each tool implements `Enchantable` on its own.

Pros	Cons
KISS: no new abstract class.	DRY: the same storage and menu code three times, the "duplicate code" smell.
Each tool reads alone, top to bottom, without a parent class to open.	A change to how `EnchantAction`s are offered needs three edits: Divergent Change, and Connascence of Algorithm between three copies.

Chosen: `AbstractEnchantableTool extends Item implements Enchantable`. It owns the offered Enchantments, the attached one (through `EnchantmentSlot`, below), `enchant()` and the menu. `Hoe`, `WateringCan` and `Pickaxe` extend it and write only `triggerEnchantment()`.

Pros	Cons
DRY: storage and the menu are written once.	A tool that should offer only some Enchantments must override `allowableActions()`; no tool needs this today, so we did not build it (YAGNI).
SRP: each tool changes only when its own spread changes.	Subclasses reach the slot through the `protected` accessor `enchantmentSlot()`, a Connascence of Name with the parent that a rename would break.
LSP: any tool works in `EnchantAction`, because every tool inherits the same `enchant()`.	

An interface default method cannot hold a field, so sharing the stored Enchantment needs a class. This is a language limit, not a style choice.

One slot that stores, fires and clears.

Firing is the same for every tool: if an Enchantment is attached, apply it to the affected tiles, then clear it. What differs is which tiles the tool's own spread touches.

Rejected: write the store, fire and clear steps inside each tool's `triggerEnchantment()`.

Pros	Cons
KISS: one class fewer.	SRP: one method mixes the bookkeeping with the tool's spread, so it changes for two reasons.
Everything about one tool's turn sits in one method, easy to read once.	DRY: the "check, apply, clear" steps three times; forgetting "clear" in one copy lets an Enchantment fire forever (Connascence of Execution spread over three classes).

Chosen: `EnchantmentSlot`, a small helper that holds at most one Enchantment and offers `enchant()`, `isEmpty()` and `fire(tiles, activator, origin)`, which applies the effect and clears itself.

Pros	Cons
SRP: each tool reads as "build my list of tiles, hand it to the slot".	One more step to follow when tracing a turn (tool, then slot, then Enchantment), close to a Middle Man for readers who expect the tool to do it.
The same role as other small helpers in the codebase, such as `StatisticAdjuster` and `MerchantContext`.	The slot holds `null` while empty; this stays private and callers only see `isEmpty()`.

Connascence: subclasses depend on the parent only by Name (`enchantmentSlot()`), never on its fields; "apply, then clear" is Connascence of Execution, kept inside one method. **Code smells:** avoided Duplicate Code and Divergent Change. Accepted: one extra level of indirection (tool, slot, Enchantment). **Foresight:** a fourth enchantable tool writes only its `triggerEnchantment()`.

C. Enchantment effects: reuse, and one contract

Each Enchantment reuses an existing effect, and all of them report in the same way.

Reusing existing classes.

Enchantment	Reuses	Instead of
`BlazeEnchantment`	The existing `Fire` item	A new burning mechanism
`ShatterEnchantment`	The existing `StunnedStatus`	A new stun status
`BountyEnchantment`	REQ1's public `Plant.attemptGrowth()`, which tries the plant's abilities without ageing it or printing	A copy of the plant's ability loop

Option	Pros	Cons
Each Enchantment writes its own fire, stun and growth logic	Each Enchantment class stands alone and can be read without other packages.	DRY: a second burning rule and a second growth loop to keep in step with the originals (Duplicate Code, Connascence of Algorithm).
Reuse the existing classes (chosen)	DRY: each effect already exists and is already tested. KISS: no parallel mechanism.	`game.enchantments` depends on `game.items`, `game.status` and `game.plants` (Connascence of Name across packages, on stable public APIs); there is no cycle.

The empty-string contract.

`Enchantable.triggerEnchantment`, `EnchantmentSlot.fire` and `Enchantment.applyTo` never return `null`. They return `""` when nothing fired, the same rule REQ1's abilities use.

Option	Pros	Cons
Return `null` for "nothing happened"	Common in Java, and cheap to write.	Connascence of Meaning: every caller must remember what `null` means, and a forgotten check crashes the turn; scattered `null` checks are a known smell.
Throw when nothing fired	Impossible to ignore, so no caller forgets the case.	An empty slot is normal, not an error; every caller would need a `try/catch` (Connascence of Execution).
Return `""` (chosen)	KISS: every caller simply joins the strings. LSP: all implementations keep one contract.	A reader must know that `""` means "nothing happened"; the Javadoc of each method says so.

Every message still names a location: when there are no affected tiles, each Enchantment reports "near" the origin, and both actions start their message with "At" plus the location.

Connascence: Connascence of Name on three stable public methods (`Fire`, `StunnedStatus`, `Plant.attemptGrowth`), and one documented Connascence of Meaning (`""`) shared with REQ1 instead of a different convention per method. **Code smells:** avoided Duplicate Code, parallel mechanisms for the same effect and scattered `null` checks.

D. Visibility, `final`, `protected` and `static`

Choice	Where	Why	Cost
Every class is `public`	All REQ4 classes	Another package can reuse them, for example a new action that uses `EnchantmentTrigger`.	A wider public API, so more classes could depend on them (more connascence across packages).
Every field is `private`	All classes	Subclasses use `enchantmentSlot()`, not the field, so no Data Class is manipulated from outside.	—
`protected` accessor	`AbstractEnchantableTool.enchantmentSlot()`	Only the tools need the slot to fire it; callers use `Enchantable`.	Classes in `game.items.tools` could also call it. None does.
`protected` constructor	`AbstractEnchantableTool`	It is abstract; only tools construct it.	—
`final` classes	`EnchantmentSlot`, `EnchantmentTrigger`	Small helpers used by composition, not by inheritance. A subclass could break "fire once, then clear".	A variant slot, such as one holding two Enchantments, would be a new class.
Constants are `private static final`	e.g. `CHAIN_BREAK_CHANCE_PERCENT`, `STUN_DURATION_TURNS`	A name explains each number, avoiding magic numbers (Connascence of Meaning); no public constants (A2 feedback).	Tests repeat the numbers.
No static methods	—	A2 feedback penalised static methods.	—

Connascence: `private` fields keep connascence on state inside each class. **Code smells:** avoided public attributes, magic numbers and missing modifiers.

E. Changes to the earlier design

Change	Why	Cost
`Hoe`, `WateringCan` and `Pickaxe` extend `AbstractEnchantableTool` instead of `Item`.	They are the three retrofitted classes (Section B).	An unenchanted tool behaves as before; its menu gains the enchant options.
`CapabilityToolAction` and `BreakAction` call `EnchantmentTrigger` after their own effect.	So every tool use can fire its Enchantment (Section A).	One more call in each action.
`TillingTool` and `WateringTool` declare `canTill(Tillable)` and `canWater(Waterable)`.	A2 feedback: empty marker interfaces are not good OO design.	Every tool returns `true` today.

Change	Why	Cost
`Enchantment` and `EnchantAction` live in `game.items.tools`.	Keeping them there removes a package cycle between `game.items.tools` and `game.enchantments`.	The interface sits beside the tools, not beside its implementations.
The tools receive their Enchantments through the constructor.	No hard-coded list inside the tools; `StackQueueValleyWorld` decides what is offered.	One more constructor parameter.

F. Known minor violations and why we accept them

Violation or cost	Why it is acceptable
`EnchantmentTrigger.fire` takes `Object usedTool`, so a wrong argument fails silently (no match, `""`) instead of at compile time.	The two callers hold tools of different types (`T extends HasStaminaCost` and `BreakingTool`), and `HasStaminaCost` does not say the tool is an `Item`. A narrower type needs a new shared interface or a new generic bound on `CapabilityToolAction`, only to tighten one parameter (YAGNI).
The three `triggerEnchantment()` bodies share a shape: check the slot, build a tile list, fire, join the messages.	Their spreads differ in kind: every adjacent tile, one random tile, or one random tile behind a chance roll. What they share, the slot logic, is already in `EnchantmentSlot`. With three different shapes, a common template would be a guess (rule of three).
`game.enchantments` depends on three other packages (Section C).	Reuse is the point; there is no cycle.

Summary

Principle	Where it shows	Change it absorbs	Where it is only partly met, and why
SRP	Offering (`AbstractEnchantableTool`), storing and firing (`EnchantmentSlot`), finding the used tool (`EnchantmentTrigger`), each spread (the tools) and each effect (the Enchantments) each have one class.	A new firing rule edits only `EnchantmentSlot`.	—
OCP	A new tool or Enchantment is one new class.	A Scythe or a Frost Enchantment: no existing class changes.	New Enchantments are listed in `StackQueueValleyWorld`, the composition root.
LSP	Any `Enchantable` works in both actions; any `Enchantment` works in any slot.	A new tool fires with no new branch.	—
ISP	`Enchantable` has two methods and `Enchantment` one; the actions see only `Enchantable`.	—	—
DIP	The actions depend on `Enchantable`, the tools on `Enchantment`; `RandomUtils` and `AreaFactory` are injected.	A new Enchantment needs no change to the tools.	`EnchantmentTrigger` is created with `new` in each action; it has no state and one possible behaviour.

DRY and KISS. The menu, the slot and the lookup each exist once, and the Enchantments reuse `Fire`, `StunnedStatus` and `Plant.attemptGrowth`. **Connascence.** Between packages it is only of Name and Type. The one Connascence of Identity (`==`) stays inside one method.

Testing. `HoeTest`, `WateringCanTest`, `PickaxeTest`, `EnchantmentSlotTest` and one test class per Enchantment cover each tool's own spread, tools with and without an Enchantment, Enchantments with and without affected tiles, and that an Enchantment is spent after one use. `FixedRandom` makes them repeatable, and `mvn test` runs them all.

Jingyu Zhang 37033956, Yuanshu Liu 36226173, Poyee Tan 36641162 – REQ 5: The Wanderer (API, HD)

The Wanderer's tile is turned into a real latitude and longitude, and OpenWeatherMap's weather there decides what it does. Every 5 turns, today's weather triggers a `WeatherReaction` (storm shelter, rain dance or campfire). A Farmer next to it can ask for tomorrow's forecast, which brings a `ForecastOmen` (frost, sowing or drought).

A. The game asks an interface, and "no reading" is a normal answer

The Wanderer never talks to OpenWeatherMap directly, and a failed request must not stop the game.

Keeping the game independent of the weather API.

Rejected: `Wanderer` and `AskForecastAction` build the URL, send the request and read the JSON themselves.

Pros	Cons
KISS: one interface and one class fewer to read.	DIP: game classes would depend on `HttpClient` and on OpenWeatherMap's JSON shape, a volatile third-party detail (Connascence of Algorithm across packages).
The whole weather path sits in one place.	Every test of the Wanderer would need the network or a fragile HTTP mock, so tests would not be repeatable.
	Changing the provider would edit `Wanderer` and `AskForecastAction` (Shotgun Surgery).

Chosen: a `WeatherSource` interface with `currentWeatherAt(lat, lon)` and `tomorrowWeatherAt(lat, lon)`, both returning `Optional<WeatherReading>`. `OpenWeatherClient` is the real one and is injected through the constructor. Tests inject `FixedWeatherSource`.

Pros	Cons
DIP: game classes depend on an interface the game owns, not on the HTTP library.	One more interface and one more class to learn before the feature makes sense.
LSP: `FixedWeatherSource` replaces the real client, so no REQ5 test uses the network.	The interface must stay general enough for any provider, so it only carries the four values the game uses; extra data needs a change to `WeatherReading`.
OCP: another provider, or a cached one, needs no change to `Wanderer`, `AskForecastAction` or `WeatherInterpreter`.	

The package `game.weather` imports nothing from the rest of the game, so it could be reused by any feature. **Foresight:** a weather source that replays saved readings for a demo is one new class.

When the sky cannot be read.

With no key, no network or a bad reply, there is no reading.

Option	Pros	Cons
Throw a checked exception	Conventional Java, and the compiler forces a handler.	Every caller needs a `try/catch` for a normal, expected case; the handler code repeats in each caller (Duplicate Code).
Return `null`	KISS: short, with no wrapper type.	A forgotten check crashes the game, and scattered `null` checks are a known smell. The rule "`null` means no reading" is invisible to the compiler (Connascence of Meaning).
Return `Optional<WeatherReading>` (chosen)	LSP: every source keeps the same contract, and "no reading" is part of the return type, so a caller sees it.	Call sites use `isEmpty()` and `get()`; a caller that calls `get()` without checking still fails, so the rule moves from Meaning to Type but is not removed.

When there is no reading, `Wanderer` wraps its normal wandering in `UnreadableSkyAction`, which reports "The sky is unreadable, and the Wanderer only wanders." `AskForecastAction` reports that tomorrow's weather is

unreadable. The game never stops, and the player always sees what happened. We use `Optional` only with `isEmpty()` and `get()`, with no chained calls.

Connascence: Connascence of Algorithm with OpenWeatherMap's JSON layout stays inside `game.weather`; the rest of the game shares only Connascence of Type on `WeatherSource`. Returning `Optional` avoids the Connascence of Meaning a `null` result would need. **Code smells:** avoided Shotgun Surgery (changing provider would otherwise edit game classes), network calls in tests and scattered `null` checks.

B. The API layer: place, key and reply each have one class

Behind `WeatherSource`, three small classes each own one part of a request: where to ask, which key to send, and how to read the reply.

Turning the Wanderer's tile into a real place.

Rejected: always ask about one fixed place, such as Melbourne.

Pros	Cons
KISS: trivial, and the same answer for every tile is easy to test.	Breaks the brief's rule that the request cannot be a static URL. The game state would not affect the query, and the coordinates would be magic numbers in a string.

Chosen: `MapGeography` turns a tile's (x, y) into a latitude and longitude from an origin (Melbourne, -37.81, 144.96) and a scale (2 degrees per tile). `Wanderer` and `AskForecastAction` ask it about the tile the Wanderer stands on, so every move changes the query. Latitude stops at the poles and longitude wraps past 180, so any tile gives a valid place.

Pros	Cons
The query is driven by the game state, as the brief requires.	One more small class, and its constructor takes three `double`s in a fixed order (Connascence of Position).
SRP: one class knows how map tiles relate to the real world. Moving the farm changes only the constructor arguments.	
`latitudeOf` and `longitudeOf` name the two numbers, so callers cannot swap them (less Connascence of Position).	Two degrees per tile puts many tiles over the sea, so many readings come from ocean weather; the scale is one constant to change.

Keeping the API key out of the code.

Rejected: `OpenWeatherClient` calls `System.getenv("OPENWEATHER_API_KEY")` itself.

Pros	Cons
KISS: one class fewer, and one line to read the key.	SRP: the client would also decide where secrets live, so a change of key location edits the network class (Divergent Change).
	A test could not control the key without changing the real environment, so tests would depend on external state.

Chosen: `ApiKeyLoader` takes the key's name, an environment map and a `.env` path. It checks the environment first, then a `NAME=value` line in `.env`, and returns `Optional<String>`. The client receives a loader through its constructor.

Pros	Cons
SRP: the loader finds the key; the client calls the API.	One more class and one more constructor parameter for the client.
A test passes a plain map and a temporary file.	The key's name must match in the loader, the `.env` file and the README (Connascence of Name across files); `StackQueueValleyWorld` holds it once.
Security: the key is never in the source, and `.env` is in `.gitignore`.	

Reading the reply.

Rejected: read the JSON inside `OpenWeatherClient`.

Pros	Cons
KISS: one class fewer; the request and its reply sit together.	SRP: the client would both talk to the network and know the JSON layout. A layout change and a network change would edit the same class (Divergent Change).

Chosen: `OpenWeatherParser` turns a reply into a `WeatherReading`. The weather condition and the temperature are required; wind and rain default to 0, because OpenWeatherMap leaves out `rain` on dry days. For the forecast it reads the last of nine 3-hour steps (about 24 hours ahead) and divides `rain.3h` by 3 to get an hourly amount. A reply missing a required field gives no reading. `WeatherReading` is a small value object: its fields are `private final`, and its constructor rejects a blank condition and negative wind or rain.

Pros	Cons
SRP: the parser is the only class that knows the JSON layout, and it is tested with saved JSON text, with no network.	Gson is an extra library, added through Maven; the parser still knows the field names (Connascence of Algorithm with the API format, kept in one class).
A bad reading cannot exist, so the reactions need no checks of their own; `WeatherReading` is not a passive Data Class, because it validates itself.	A reply that is valid JSON but holds odd values, such as a temperature of 500°C, is accepted; we only reject what the game cannot use.

Connascence: `latitudeOf` and `longitudeOf` name the two numbers, reducing Connascence of Position, though the three-`double` constructor still has it (Section G). The key's name is Connascence of Name, written in one place, and Connascence of Algorithm with the API's format is confined to the parser. **Code smells:** avoided a static URL, magic numbers, a secret in the source code, Divergent Change in the client, Primitive Obsession (four loose numbers) and a passive Data Class, because `WeatherReading` validates and describes itself.

C. From a reading to an effect: one interpreter, two interfaces

A reading must become a reaction today or an omen tomorrow, without the Wanderer knowing the rules.

Choosing which reaction today's weather triggers.

Option	Pros	Cons
An if-chain inside `Wanderer.playConsciousTurn` that also performs each effect	KISS: no extra class; the rules sit where they are used.	SRP: the actor would also hold every weather rule. OCP: a new reaction edits `Wanderer`. The bare numbers would need a reader to guess their meaning (Connascence of Meaning).
A configurable rule engine: a list of (condition, effect) pairs, perhaps read from a file	OCP: rules could change without recompiling.	The six rules are fixed by the proposal. A generic list is harder to read than a few `if`s and solves a problem we do not have: the Speculative Generality smell (YAGNI, KISS).
`WeatherInterpreter` with `reactionFor(reading)` (chosen)	SRP and DIP: `Wanderer` sees only `WeatherReaction`. DRY: each threshold is a named constant that exists only here.	OCP is only partly met: a new rule adds one `if` here, and this one class names all six concrete classes (Connascence of Name with each).

`WeatherInterpreter` is the only class that creates the six concrete classes. It works as a simple factory. `CampfireReaction` and `FrostOmen` size their effect by how far the temperature is below the threshold that chose them, so `WeatherInterpreter` passes that threshold into their constructors. Our first version repeated 10°C and 2°C as constants in those two classes (Connascence of Value across classes): changing a threshold in one place would have left the effect sized against the old number. Passing it in leaves one source for each number. **Foresight:** a `HeatwaveReaction` above 35°C is one new class and one new `if` in `WeatherInterpreter`; `Wanderer` and the other reactions do not change.

Two interfaces, not one.

Rejected: one interface for both sides, with `reactAround(Location)` and `bringUpon(Actor, Location)`.

Pros	Cons
KISS: one interface to learn for every weather effect.	ISP: every reaction would carry an omen method it cannot use, and every omen a reaction method. The unused bodies would do nothing or throw, so a caller could not trust every implementation (LSP risk). This is the Refused Bequest smell.

Chosen: `WeatherReaction` (`reactAround(Location)`) and `ForecastOmen` (`bringUpon(Actor, Location)`). Both are chosen in the same `WeatherInterpreter`, because "which effect does this reading trigger" is one job for today's and tomorrow's weather.

Pros	Cons
ISP: each class implements only the method it needs.	Two interfaces that look alike, which a reader must learn apart; a new kind of effect may need a third.
DIP: `AskForecastAction` depends only on `ForecastOmen`; `Wanderer` and `ReactToWeatherAction` only on `WeatherReaction`.	
DRY: `isRainy` and the condition names are shared by both methods inside one class.	Both sides depend on `WeatherReading` (Connascence of Type), so a new field there touches both.

Connascence: named constants avoid Connascence of Meaning; passing the thresholds in removed Connascence of Value between the interpreter and two effect classes; each action has Connascence of Type with one interface only. **Code smells:** avoided Switch Statements inside `Wanderer`, magic numbers, Speculative Generality (a configurable rule engine) and Refused Bequest (reactions forced to implement an omen method). Accepted: one factory class names the six concrete classes.

D. Letting the Farmer ask about tomorrow

Rejected: give `Farmer` a method that searches the map for a Wanderer and asks it.

Pros	Cons
KISS: no new action class.	"Feature envy" smell: the data (`WeatherSource`, `MapGeography`, `WeatherInterpreter`) belongs to the Wanderer, not the Farmer.
The Farmer's options sit in the Farmer's class.	Any other actor that wants to ask would copy the same search (DRY), and each copy would be Connascence of Algorithm with the others.

Chosen: `Wanderer.allowableActions` offers an `AskForecastAction` to whoever stands next to it. This is the engine's normal way for one actor to offer an action to another. `ReactToWeatherAction` and `AskForecastAction` only pass the turn on to the reaction or omen they hold.

Pros	Cons
DRY: reuses the engine's protocol, like every other interactive entity.	Any adjacent actor is offered the action (Section G), and `AskForecastAction` holds the same three collaborators as `Wanderer` (a small Data Clump).
SRP: each action has one job, and neither names a concrete class.	`ReactToWeatherAction` only passes the turn to its reaction, close to a Middle Man; the engine needs an `Action` object, so it stays.

The engine only asks an actor that is on the map for its actions, so `AskForecastAction` never needs to check that the Wanderer is still there.

Connascence: Only Connascence of Name with the engine's existing API. **Code smells:** Avoided: Feature Envy (a Farmer method using the Wanderer's data). Accepted: `ReactToWeatherAction` and `UnreadableSkyAction` are close to Middle Men, because the engine needs an `Action` object.

E. Visibility, `final`, `protected` and `static`

Choice	Where	Why	Cost
Every class is `public`	All REQ5 classes	`game.weather` is meant to be reused by other features.	A wider public API, so more classes could depend on them (more connascence across packages).

Choice	Where	Why	Cost
Every field is `private`, and `final` where it never changes	All classes; `WeatherReading` is fully immutable	A reading is shared by the interpreter and the effect it creates, so it must not change.	`Wanderer.turnsSinceLastReaction` is the only changing field.
`protected` method	`OpenWeatherClient.fetch(URI)`	The one seam a test overrides to return saved replies without the network.	A subclass could replace the network call. The class comment says it exists for tests.
`private` helpers	e.g. `OpenWeatherParser.readingFrom`, `WeatherInterpreter.isRainy`	They are steps, not contracts.	—
Constants are `private static final`	e.g. `STORM_WIND_THRESHOLD_MPS`, `TIMEOUT`	A name explains each number, avoiding magic numbers (Connascence of Meaning); no public constants (A2 feedback).	Tests repeat the numbers.
No static methods	—	A2 feedback penalised static methods.	—

Connascence: `private` fields keep connascence on state inside each class; `WeatherReading` is immutable.
Code smells: Avoided: public attributes and missing modifiers.

F. Changes to the earlier design

REQ5 is new in A3. It builds on earlier classes without changing them, except `StackQueueValleyWorld`.

Earlier class	How REQ5 uses it
`WanderingActor` (A2)	`Wanderer` extends it, so the unconscious check and wandering are reused.
`TemporaryGround` (REQ3)	`StormShelterGround` extends it, so the shelter restores the old ground with no new countdown code.
`Fire` (A2)	`CampfireReaction` lights the same `Fire` item.
`GrowthPausedStatus` (REQ1), `TimedStatus` (A2)	`FrostOmen` pauses plants; `DroughtWearinessStatus` reuses the countdown.
`Plantable`, `StrawberrySeedling` (REQ1)	`SowingOmen` plants through the same rule as everything else.
`StackQueueValleyWorld`	Builds the client, the key loader and `MapGeography`, and places one Wanderer.

G. Known minor violations and why we accept them

Violation or cost	Why it is acceptable
The Wanderer can be burnt by its own campfire if it later walks onto a burning tile.	On purpose: `Fire` burns any actor and never asks who lit it, so it needs no type check. A Wanderer that stays somewhere cold pays for its fires, and it has 50 hit points.
`OpenWeatherClient` creates its own `HttpClient`, a small DIP gap.	Tests replace `fetch`, not the `HttpClient`, so injecting it would add a parameter nobody varies.
The weather condition is a `String` such as `"Rain"`, so the code depends on OpenWeatherMap's names (Connascence of Meaning).	The names are named constants in `WeatherInterpreter`, and the parser is the only class that reads them from JSON.

Violation or cost	Why it is acceptable
`AskForecastAction` holds the same three collaborators as `Wanderer` (a small "data clump").	They are passed once, in `allowableActions`. A holder class for three fields would be a class used once.
Any actor next to the Wanderer is offered `AskForecastAction`, not only the Farmer.	Only the player picks menu actions; other actors never choose it.
`WeatherInterpreter` names all six concrete classes, so a new reaction edits it. OCP is only partly met.	It is the one factory point (Section C). Everything else stays closed.
`ApiKeyLoader` and `OpenWeatherParser` turn errors into an empty result.	The game must keep running without weather. Tests cover the missing key, the missing file and the broken JSON.

Summary

Principle	Where it shows	Change it absorbs	Where it is only partly met, and why
SRP	The API call (`OpenWeatherClient`), the JSON (`OpenWeatherParser`), the key (`ApiKeyLoader`), the place (`MapGeography`), the rules (`WeatherInterpreter`) and each effect each have one class.	A new JSON layout edits only the parser.	—
OCP	A new reaction, omen or weather provider is one new class.	A Heatwave reaction or a cached weather source.	`WeatherInterpreter` gains one `if` per new rule (Section G).
LSP	Any `WeatherReaction` or `ForecastOmen` works in its action; any `WeatherSource` works in the Wanderer.	`FixedWeatherSource` replaces the real client in every test.	—
ISP	`WeatherReaction` and `ForecastOmen` each have one method; `WeatherSource` has the two the Wanderer needs.	—	—
DIP	`Wanderer` depends on `WeatherSource` and `WeatherReaction`; `AskForecastAction` on `ForecastOmen`; the client on `ApiKeyLoader`.	A different provider needs no change to the game.	`OpenWeatherClient` creates its own `HttpClient` (Section G).

DRY and KISS. The rules live in one interpreter, the countdowns reuse `TemporaryGround` and `TimedStatus`, and the effects reuse `Fire`, `GrowthPausedStatus` and `Plantable`. Six fixed rules are a few `if`s, not a rule engine.
Connascence. Between packages it is only of Name and Type, through `WeatherSource`, `WeatherReaction` and `ForecastOmen`. Each threshold exists once, in `WeatherInterpreter`, which passes the cold and frost thresholds to the classes that size their effect by them.

Testing. Each reaction, omen, action and API-layer class has its own test class, for example `WeatherInterpreterTest`, `WandererTest`, `OpenWeatherParserTest` and `ApiKeyLoaderTest`. They cover readings that meet or miss each rule, the priority between rules, and invalid input such as a missing key, broken JSON and negative wind. No test uses the network: `FixedWeatherSource` replaces the client, and `OpenWeatherClientTest` overrides `fetch`. `mvn test` runs them all. Separately, `WandererStressHarness` is a manual tool, not part of `mvn test`: it plays many seeded games through the real classes with fake weather to look for rare crashes.